

ระบบให้บริการแผนที่บนเว็บ

นายเกียรติศักดิ์ คิวขุนทด 43015352
นายณรินทร์ เรืองศรี 43015364
ดร. วิศิษฐ์ หิรัญกิตติ อาจารย์ที่ปรึกษา
ปีการศึกษา 2545

บทคัดย่อ

แผนที่เป็นสิ่งจำเป็นอย่างยิ่งสำหรับการเดินทาง แผนที่คอมพิวเตอร์ที่มีให้บริการบนเว็บในปัจจุบันสามารถให้บริการข้อมูล เส้นทาง สถานที่ ระบบสามารถให้บริการสอบถามถนน สถานที่สำคัญว่าอยู่ตำแหน่งใดบนแผนที่ แต่ยังไม่สามารถให้บริการสอบถามเส้นทางการเดินทางจากจุดหนึ่งไปยังอีกจุดหนึ่งที่สิ้นสุด

ปริญญานิพนธ์นี้ศึกษาหลักการทางปัญญาประดิษฐ์เพื่อใช้สร้างฐานความรู้แผนที่เพื่อให้บริการแผนที่บนเว็บโดยใช้ภาษาโปรล็อก และเพื่อให้สามารถให้บริการความรู้แผนที่บนเว็บจึงได้เลือกใช้โปรเว็บ(ProWeb) ซึ่งเป็นภาษาโปรล็อกสำหรับใช้พัฒนาโปรแกรมประยุกต์บนเครื่องเว็บเซิร์ฟเวอร์ สำหรับส่วนฐานข้อมูลของแผนที่นั้นพัฒนาโดยอาศัยโปรคาด้า (ProData)

Map Service on the Web

Keattisak Kewkhunthod 43015352

Narin Ruangsri 43015364

Dr. Visit Hirankitti Adviser

ABSTRACT

Maps are essential elements for traveling. Current web-based map services can provide positions of roads and places. The users can ask for the location of particular roads and places on the map.

In this thesis we employ artificial intelligence for developing knowledge-based map service on the web using Prolog. We use ProWeb an extension of Prolog to develop the web application on the web server and ProData to connect the system databases.

กิตติกรรมประกาศ

ปริญญานิพนธ์ฉบับนี้คงไม่อาจสำเร็จได้ด้วยดี หากไม่ได้รับความช่วยเหลือ และร่วมมือจากหลายๆฝ่ายด้วยกัน บุคคลแรกที่จะขาดเสียไม่ได้ในความสำเร็จดังกล่าวนี้ คือ อาจารย์ วิศิษฐ์ หิรัญกิตติ อาจารย์ที่ปรึกษาปริญญานิพนธ์ที่ไม่เพียงแต่สละเวลาเท่านั้น แต่ยังสละแรงกายและแรงใจในการเอาใจใส่ดูแล แนะนำ และช่วยเหลือเสมอมาจนปริญญานิพนธ์ฉบับนี้สำเร็จลุล่วงด้วยดี อีกทั้งคณาจารย์ทุกท่านท่านที่ถ่ายทอดวิชาความรู้มาให้ ซึ่งในความกรุณาของอาจารย์ที่มีต้องขอบคุณเป็นอย่างสูง

นอกจากนี้ต้องขอบคุณเจ้าหน้าที่บริษัท Logic Programming Associates ที่ช่วยให้คำปรึกษาเกี่ยวกับโปรแกรม WIN-PROLOG และต้องขอบพระคุณบุคคลสำคัญที่สุดที่ทำให้วันนี้ ก็คือ บิดา มารดา อันเป็นที่เคารพรักยิ่งซึ่งเลี้ยงดูมาเป็นอย่างดี พร้อมทั้งโอกาสในการศึกษาอย่างเต็มที่ และยังให้กำลังใจ เอาใจใส่เสมอมาในทุกๆด้าน อันที่หาที่เปรียบมิได้ ข้าพเจ้าขอระลึกในพระคุณ และขอกราบขอบพระคุณเป็นอย่างสูงมา ณ ที่นี้

นายเกียรติศักดิ์ กิวขุนทด

นายณรินทร์ เรืองศรี

สารบัญ

	หน้าที่
บทคัดย่อภาษาไทย	I
บทคัดย่อภาษาอังกฤษ	II
กิตติกรรมประกาศ	III
สารบัญ	IV
สารบัญภาพ	VII
สารบัญตาราง	IX
บทที่ 1 บทนำ	
1.1 ความสำคัญและที่มา	1
1.2 ขอบเขตปริญญานิพนธ์	1
1.3 วิธีการดำเนินงาน	2
บทที่ 2 ภาษาโปรแกรม	
2.1 ภาษาโปรแกรม	3
2.2 หลักการขั้นต้นของภาษาโปรแกรม	3
2.3 ข้อเท็จจริง (Fact)	5
2.4 ตัวแปรในโปรแกรม	6
2.5 ตัวแปรที่ไม่ต้องการระบุชื่อ	7
2.6 กฎ (Rule)	7
2.7 ความจริงบางประการในการทำงานของโปรแกรม	8
2.8 การย้อนกลับ (Backtracking)	9
2.9 เพรดิกเทฟ	11
2.10 เพรดิกเทคท์	11
2.11 การตัดสินใจในการใช้เพรดิกเทคท์	11
บทที่ 3 โปรแกรม	
3.1 โปรแกรมเวิร์กเวอร์	13
3.1.1 การทำงานย้อนกลับบนเว็บ	13
3.1.2 การให้บริการผู้ใช้งานจำนวนมาก	13
3.1.3 กระบวนการทำงานของโปรแกรมเวิร์กเวอร์	14
3.2 การติดตั้งและตั้งค่าโปรแกรมเวิร์กเวอร์	15
3.2.1 ความต้องการของระบบ	15
3.2.2 โปรแกรมเวิร์กเวอร์	15

3.2.3 การติดตั้งโปรเว็บ	15
3.2.4 การสร้างไฟล์ PROWEB.SYS	16
3.3 การใช้งานโปรเว็บ	17
3.4 โปรคาต้า	18
3.5 การติดตั้งโปรคาต้า	19
3.6 การใช้งานโปรคาต้า	21
3.6.1 การเชื่อมโปรล็อกกับโปรคาต้า	21
3.6.2 การเชื่อมกับฐานข้อมูล	21
3.6.3 การใช้งานตารางในฐานข้อมูล	21
3.6.4 การใช้คำสั่งคิวรีโดยตรง	22
บทที่ 4 ระบบแผนที่และการวินิจฉัย	
4.1 ระบบแผนที่ (Map System)	23
4.2 แผนที่ดิจิทัล	23
4.3 ระบบพิกัดบนแผนที่	25
4.3.1 การคำนวณระยะทางบนแผนที่	25
4.3.2 ระยะทางระหว่างจุดสองจุดบนแผนที่	25
4.3.3 ระยะทางระหว่างจุดถึงเส้นตรง	26
4.4 ส่วนการวินิจฉัย	26
4.5 การค้นหาประเภท Uniformed State Space Search	27
4.5.1 การค้นหาตามแนวลึกก่อน (Depth First Search)	27
4.5.2 การค้นหาตามแนวกว้างก่อน (Breadth First Search)	28
4.6 การค้นหาประเภท Informed State Space Search (Heuristic Search)	28
4.6.1 การค้นหาแบบกรี้ดี (Greedy Search)	29
4.6.2 การค้นหาแบบฮิวริสติกแบบเอ-สตาร์ (A*)	30
4.6.3 การพิสูจน์เส้นทางที่ดีที่สุดของเอ-สตาร์	32
4.7 การค้นหาเส้นทางบนแผนที่	33
4.8 การค้นหาเส้นทางที่ดีที่สุดโดยประยุกต์ใช้วิธีเอสตาร์	33
บทที่ 5 การออกแบบและการสร้างเว็บไซต์	
5.1 การออกแบบ	34
5.2 การสร้างเว็บไซต์	34
5.2.1 โปรเว็บ	34
5.2.2 ส่วนวินิจฉัย	35
5.2.3 ฐานข้อมูล	37
5.2.4 Servlet	39

5.2.4.1 เหตุผลการใช้งาน Servlet	39
5.2.4.2 การทำงาน	40
5.2.4.3 ปัญหาในการสร้าง	40
บทที่ 6 ผลการทดลอง	
6.1 การทดลองย่อ-ขยาย และเลื่อนแผนที่	41
6.2 การค้นหาถนนสายหลัก	43
6.3 การค้นหาสถานที่สำคัญ	44
6.4 การค้นหาสถานที่ใกล้เคียง	46
6.5 การค้นหาเส้นทางที่สั้นที่สุด	48
6.6 วิจารณ์ผลการทดลอง	49
บทที่ 7 บทวิจารณ์และสรุป	
7.1 บทสรุป	50
7.2 ปัญหาที่เกิดขึ้น	50
7.3 วิธีการแก้ปัญหา	50
7.4 แนวทางการพัฒนา	51
ภาคผนวก ก. Java Script & Servlet	52
ภาคผนวก ข. การติดตั้งเว็บไซต์ระบบให้บริการแผนที่บนเว็บ	54
บรรณานุกรม	60

สารบัญภาพ

	หน้าที่
รูปที่ 2-1 ตัวอย่างแสดงการย้อนกลับ (Backtracking)	10
รูปที่ 3-1 รูปบล็อกไดอะแกรมการทำงาน	14
รูปที่ 3-2 ทดสอบการทำงานของ Apache	16
รูปที่ 3-3 ผลลัพธ์โปรแกรม Hello World	17
รูปที่ 3-4 รูปโค้ดโปรแกรม Hello World	17
รูปที่ 3-5 รูปโค้ด HTML ของโปรแกรม Hello World	18
รูปที่ 3-6 ODBC Data Sources Administrator	19
รูปที่ 3-7 เลือก Driver สำหรับการสร้าง Data Source	20
รูปที่ 3-8 การสร้าง Data source โดยใช้ Access	20
รูปที่ 4-1 ตัวอย่างแผนที่ดิจิทัล	23
รูปที่ 4-2 การแสดงผล แบบราสเตอร์ และแบบเวกเตอร์	24
รูปที่ 4-3 ตัวอย่างของแผนที่แบบราสเตอร์ และแบบเวกเตอร์	24
รูปที่ 4-4 ระบบพิกัด	25
รูปที่ 4-5 การคำนวณระยะห่างระหว่างจุดและเส้นตรงกับจุด	26
รูปที่ 4-6 การค้นหาแนวเล็ก	27
รูปที่ 4-7 การค้นหาแนวกว้าง	28
รูปที่ 4-8 การค้นหาแบบกริด	29
รูปที่ 4-9 การเลือกเส้นทางโดยการพิจารณาจาก $h(n)$	29
รูปที่ 4-10 ส่วนประกอบของ heuristic function $f(n)$	30
รูปที่ 4-11 การค้นหาแบบฮิวริสติก	31
รูปที่ 4-12 ลำดับการค้นหาแบบฮิวริสติก	31
รูปที่ 4-13 เปรียบเทียบการเดินทางที่ดีที่สุด	32
รูปที่ 4-14 ลักษณะของ function ต่าง ๆ	33
รูปที่ 5-1 ไดอะแกรมของเว็บไซต์	34
รูปที่ 5-2 ไดอะแกรมการทำงานของ Servlet	40
รูปที่ 5-3 ตัวอย่างการแปลงไฟล์ WMF ผ่าน Servlet	40
รูปที่ 6-1 แสดงแผนที่ขนาดย่อ	41
รูปที่ 6-2 แสดงการขยายแผนที่	42
รูปที่ 6-3 แสดงการเลื่อนแผนที่	42
รูปที่ 6-4 แสดงการค้นหาถนนสายหลัก	43

รูปที่ 6-5 แสดงการค้นหานนสายหลัก	44
รูปที่ 6-6 แสดงการค้นหาสถานที่สำคัญ	45
รูปที่ 6-7 แสดงการค้นหาสถานที่สำคัญ	45
รูปที่ 6-8 แสดงการค้นหาสถานที่สำคัญ	46
รูปที่ 6-9 แสดงการค้นหาสถานที่ใกล้เคียง	46
รูปที่ 6-10 แสดงการค้นหาสถานที่ใกล้เคียง	47
รูปที่ 6-11 แสดงการค้นหาสถานที่ใกล้เคียง	47
รูปที่ 6-12 แสดงการค้นหาเส้นทางที่สั้นที่สุด	48
รูปที่ 6-13 แสดงการค้นหาเส้นทางที่สั้นที่สุด	48

สารบัญตาราง

	หน้าที่
ตารางที่ 5 -1 ตาราง POSITION	37
ตารางที่ 5 -2 ตาราง PATH	37
ตารางที่ 5 -3 ตาราง PLACE	38
ตารางที่ 5 -4 ตาราง PLACETYPE	38
ตารางที่ 5 -5 ตาราง ROAD	38
ตารางที่ 5 -6 ตาราง ROADPATH	39
ตารางที่ 5 -7 ตาราง ROADTYPE	39
ตารางที่ 5 -8 ตาราง CROSSROAD	39

ภาคผนวก ก.

Java Script & Servlet

1. จาวา (Java)

Java คือภาษาโปรแกรมคอมพิวเตอร์ภาษาหนึ่ง ที่ได้รับการพัฒนาโดยบริษัท Sun Microsystems
กำเนิดของภาษาจาวามีสาเหตุเริ่มต้นจากความยุ่งยากในการพัฒนาโปรแกรมประยุกต์ (application) สำหรับระบบ
อินเทอร์เน็ต ทั้งนี้เพราะระบบอินเทอร์เน็ต เป็นระบบเปิดที่สามารถใช้งานจากเครื่องคอมพิวเตอร์ประเภทใด ๆ ก็ได้
ไม่ว่าจะเป็นเครื่องพีซี, แมคอินทอช, ซัน, เครื่องมินิคอมพิวเตอร์ ไปจนถึงเครื่องระดับซูเปอร์คอมพิวเตอร์ พ.ศ.
2534 บริษัท ซัน ไมโครซิสเต็มส์ (Sun Microsystems, Inc.) จึงได้พัฒนาภาษาคอมพิวเตอร์ใหม่ที่มีประสิทธิภาพใน
การทำงานชนิดไม่ยึดติดกับแพลตฟอร์มขึ้นมา

จาวาจะทำการคอมไพล์คำสั่งจากซอร์สโค้ดให้กลายเป็นรหัสภาษากลางที่เรียกว่า ไบต์โค้ด (byte code) มี
คุณลักษณะเด่นคือ มีขนาดเล็ก สามารถนำไปประยุกต์ใช้งานได้สะดวกรวดเร็ว โดยเตรียมโปรแกรมไว้บนเครื่อง
เซิร์ฟเวอร์และเมื่อมีการเรียกใช้งานจากเว็บเบราว์เซอร์ เซิร์ฟเวอร์จะทำการส่งข้อมูลโปรแกรมหากลับเพื่อให้
เว็บเบราว์เซอร์สั่งให้ทำงาน (run) ต่อไป

2. Java Script

Java Script นั้นถูกพัฒนาโดยทีมงาน Netscape เป็นโปรแกรมที่ทำงานเป็นสคริปต์เล็กๆ ใช้งานง่าย ทำงาน
โดยตัวโปรแกรมคำสั่ง อยู่ในซอร์สโค้ดของเว็บเพจ

Java Script ใช้ประโยชน์สำหรับงานด้านต่าง ๆ ทั้งการคำนวณการแสดงผล การรับ-ส่งข้อมูล และที่สำคัญ
คือสามารถโต้ตอบกับผู้ใช้ได้อย่างทันที ทั้งนี้จะช่วยให้เว็บเพจมีการเคลื่อนไหว เว็บเพจดูมีลูกเล่นมากขึ้น

การติดต่อสื่อสารระหว่างผู้ใช้งานกับเว็บเพจนั้น ต้องติดต่อโดยผู้ใช้งานส่งข้อมูลผ่านทางแบบฟอร์มที่มีการเขียน
โปรแกรม CGI รองรับไว้ในเครื่องเซิร์ฟเวอร์ เมื่อมีการส่งข้อมูลจากแบบฟอร์มมายังเครื่องเซิร์ฟเวอร์โปรแกรม
CGI จะประมวลผลข้อมูลทุกครั้ง แล้วนำผลลัพธ์ที่ได้ส่งกลับคืนไปยังเครื่องคอมพิวเตอร์ของผู้ใช้ในรูปของเว็บเพจ
HTML ใหม่ ลองนึกดูว่ากรณีที่จำเป็นต้องมีการประมวลผลในลักษณะอย่างนี้ซ้ำ ๆ กันหลาย ๆ ครั้ง CPU ที่เครื่อง
เซิร์ฟเวอร์จะทำงานซ้ำ ๆ กันมากเกินไปจนความจำเป็นด้วยเหตุนี้จึงมีการนำเอา JavaScript เข้ามาแก้ไขจุดบกพร่อง
ดังกล่าว โดยให้เครื่องคอมพิวเตอร์ของผู้ใช้ทำหน้าที่ประมวลผลเอง เพื่อช่วยลดภาระการทำงานของ CPU ที่
เซิร์ฟเวอร์

ภาษา Java Script นั้น จะคล้ายกับ ภาษาจาวาแต่เข้าใจง่ายกว่าจาวา การเขียน Java Script ก็ใช้เพียง text
editor ทั่วไป และ เว็บเบราว์เซอร์ที่สนับสนุนภาษา Java Script

โปรแกรม Microsoft Internet Explorer, Netscape Navigator และ Opera มีความ แตกต่างกันในการ
สนับสนุน Java Script เช่นตัว Object ที่อ้างอิงถึงเว็บเบราว์เซอร์ต่างกัน บางที function เหมือนกันแต่ให้ผลลัพธ์
ต่างกัน ผู้เขียน Java Script ควรจะระมัดระวังในจุดนี้ และเขียนโปรแกรมให้สามารถใช้งานได้หลายเบราว์เซอร์เลย
ยิ่งดี

ถ้าจำแนก Java Script ออกตามลักษณะการทำงานทางฝั่ง ไคลเอนต์และฝั่งเซิร์ฟเวอร์ จะจำแนกออกได้เป็น 2 แบบดังนี้

1. Navigator JavaScript เป็น Client-Side JavaScript ซึ่งหมายถึง JavaScript ที่ถูกแปลทางฝั่งไคลเอนต์ จึงมีความเหมาะสมต่อการใช้งานของผู้ใช้ทั่วไปเป็นส่วนใหญ่
2. LiveWire JavaScript เป็น Server-Side JavaScript ซึ่งหมายถึง JavaScript ที่ถูกแปลทางฝั่งเซิร์ฟเวอร์ สามารถใช้ได้เฉพาะกับ LiveWire ของ Netscape โดยตรง

โปรแกรมภาษาจาวามีความสลับซับซ้อนกว่า ต้องมีการเขียนที่รัดกุมมากกว่า Java Script ถ้าเป็นงานง่ายๆ เพื่อลดภาระ เซิร์ฟเวอร์ ก็ใช้ Java Script ในการทำงาน ได้ ถ้าเป็นงานที่มีลักษณะเฉพาะ และใช้งานมากกว่า เช่น โปรแกรม Game Online ที่ function ในฝั่งผู้ใช้เยอะมาก ต้องมีการสร้าง GUI ขึ้นมา แบบนี้ก็จะต้องใช้ Java applet หรือ ActiveX

3. Introduction to Java Servlet

Server Side Application ในระยะแรก ๆ มักถูกเขียนขึ้นด้วยคอนเซ็ปของ CGI (Common Gateway Interface) โดยหลักการทำงานง่าย ๆ ก็คือ เว็บเบราว์เซอร์ จะทำการส่งข้อมูลที่เกิดจากการกระทำของผู้ใช้ เช่น การคลิกลิงค์หรือการกรอกแบบสอบถามไปยัง เว็บเซิร์ฟเวอร์ โดยแทนที่ เว็บเซิร์ฟเวอร์ จะทำการส่งเพจที่เป็น static page กลับมาเว็บเซิร์ฟเวอร์ จะทำการส่งข้อมูลดังกล่าวไปยังโปรแกรมซึ่งถูกจัดไว้ โปรแกรมดังกล่าวจะทำการประมวลผลข้อมูลที่ได้แล้วจะส่งผลกลับไปยัง เว็บเซิร์ฟเวอร์ ซึ่งทาง เว็บเซิร์ฟเวอร์ ก็จะส่งผลที่ได้นี้กลับไปยังเว็บเบราว์เซอร์อีกทีหนึ่ง

Server Side Application ที่เป็นผลผลิตของ CGI ในรุ่นแรก ๆ ที่ยังหลงเหลืออยู่ในปัจจุบันนี้ ยกตัวอย่าง เช่น Counter, Image Map, Guess book, Send Mail เป็นต้น จริงๆแล้วตัว CGI เองสามารถเขียนด้วยภาษาอะไรก็ได้ แต่ที่นิยมมากที่สุดเห็นจะเป็น C และ Perl อาจจะเป็นเพราะว่า CGI เป็นสิ่งที่มีมากับอินเทอร์เน็ตตั้งแต่ช่วงแรก ๆ ดังนั้นไม่ว่าจะเป็น เว็บเซิร์ฟเวอร์ ใหนก็ตาม Server Side Application พื้นฐานที่ทาง เว็บเซิร์ฟเวอร์ เหล่านั้นจะต้องสนับสนุนก็คือ CGI ซึ่งจุดนี้เองที่เป็นจุดเด่นทำให้ CGI เป็นที่นิยมใช้กันอย่างกว้างขวางจนกระทั่งปัจจุบันนี้

4. Servlet

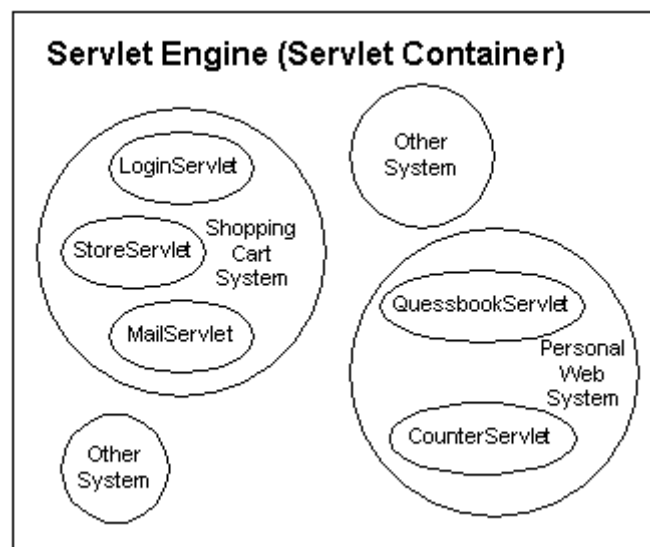
Servlet เป็น Server Side Application แบบหนึ่งซึ่งอ้างอิงคอนเซ็ปมาจาก CGI ข้อดีของ Servlet ที่อยู่นเหนือ CGI อย่างแรกก็คือตัวภาษาที่ใช้เขียนซึ่งก็คือจาวานั้นเอง จาวาเป็นภาษาที่ใช้คอนเซ็ปของ Object Oriented ในการเขียน หลายคนที่เกี่ยวข้องกับการเขียนโปรแกรมสำหรับโปรเจกใหญ่ ๆ จะทราบดีว่า Object Oriented สามารถลดความซับซ้อนของโครงสร้างโปรแกรมรวมถึงการอำนวยความสะดวกในการ reuse ส่วนของโปรแกรมที่เขียนไว้แล้วเพียงไร นอกจากนี้จาวายังเป็นภาษาที่เป็นลักษณะแบบ platform independent ซึ่งจะช่วยให้เราสามารถที่จะทำการพัฒนาระบบโดยใช้ Environment อะไรก็ได้ซึ่งโดยทั่วไปมักนิยมใช้ Window Environment โดยจะนำโปรแกรมที่เขียนเสร็จแล้วมารันบน Unix Environment เพื่อเพิ่มประสิทธิภาพของโปรแกรมแทน

นอกจากนี้ Servlet ยังมีความเร็วที่สูงกว่า CGI เพราะ Servlet ใช้หลักการของ thread โดยจะทำการสร้าง 1 thread ต่อหนึ่ง request ที่มาจาก ไคลเอนต์ ซึ่งในทางกลับกัน CGI จะทำการสร้าง 1 process ต่อหนึ่ง request ซึ่งจะ

ทำให้เปลืองทรัพยากรมากกว่าและ process ในการรันก็จะช้ากว่าด้วย ท้ายที่สุดจุดเด่นที่สำคัญของ Servlet ก็คือ API (Application Programming Interface) โดยระบบที่ทำการพัฒนาโดยใช้คอนเซ็ปของ Servlet จะสามารถเรียกใช้ API ที่ทางจาวามีมาให้ (javax.servlet.*, javax.servlet.http.*) ซึ่งจะช่วยให้การพัฒนากระบวนงานดังกล่าวง่ายและเร็วยิ่งขึ้น

5. Servlet Engine

ในการรันระบบที่เขียนขึ้นโดยใช้หลักการของ Servlet เราจะต้องนำระบบดังกล่าวมาบรรจุอยู่ในสิ่ง ๆ หนึ่งที่เรียกว่า Servlet Engine ให้นึกถึง Servlet Engine คล้าย ๆ กับกล่อง ๆ หนึ่งที่ใส่ลูกปิงปองไว้หลายลูก โดยลูกปิงปองแต่ละลูกก็คือระบบ ๆ หนึ่งนั่นเอง หลายคนอาจสงสัยทำไมถึงใช้คำว่าระบบ โดยทั่วไป Server Side Application หนึ่ง ๆ ที่ถูกเขียนขึ้นโดยใช้ Servlet API จะถูกเรียกว่า Servlet ในหนึ่งระบบอาจประกอบด้วย Servlet หลายอัน ยกตัวอย่างเช่น ระบบที่เกี่ยวกับ Shopping Cart อาจจะประกอบด้วย Servlet ที่ทำหน้าที่ในการเช็คสต็อกสินค้า, Servlet ที่ทำหน้าที่ในการเก็บข้อมูลสินค้า, Servlet ที่ทำหน้าที่ในการส่งเมลล์กลับไปยังลูกค้าเพื่อบอกว่าได้ทำการส่งของไปให้แล้ว เป็นต้น ดังนั้นถ้ามองโดยรวมแล้ว Servlet Engine ก็คือที่รวมของระบบตั้งแต่หนึ่งระบบถึงหลายระบบ โดยแต่ละระบบจะประกอบด้วย Servlet หนึ่งอันหรือมากกว่า ดังรูปที่ 1.



รูปที่ 1. Servlet Engine and its Servlets

Engine เป็นเพียงกล่อง ๆ หนึ่งที่ใส่บรรจุและรันกลุ่มของ Servlet เท่านั้น ในการที่จะทำการติดต่อสื่อสารกับ ไคลเอนต์ ตัว Servlet Engine นี้จะต้องทำงานร่วมกับ เว็บเซิร์ฟเวอร์ ซึ่งเปรียบเสมือนฉากหน้าที่ติดต่อกับ ไคลเอนต์ อีกทีหนึ่ง เมื่อใดก็ตามที่มีร้องขอมาจากไคลเอนต์ ถ้าการร้องขอนั้นเจาะจงมาที่ตัว Servlet ทางเว็บเซิร์ฟเวอร์ ก็จะทำการส่งการร้องขอนั้นมาให้ Servlet Engine ซึ่งทาง Servlet Engine ก็จะทำการเรียก Servlet ที่ไคลเอนต์ต้องการขึ้นมาทำการประมวลผลการร้องขอนั้น โดยท้ายสุด Servlet จะส่งผลกลับไปให้ Servlet Engine, Servlet Engine ก็จะส่งผลที่ได้กลับไปให้ เว็บเซิร์ฟเวอร์ ซึ่ง เว็บเซิร์ฟเวอร์ ก็จะส่งผลกลับไปให้ ไคลเอนต์

Servlet Engine อาจจะเป็นส่วนที่ติดมากับเว็บเซิร์ฟเวอร์ อยู่แล้วยกตัวอย่างเช่น Servlet Engine ที่อยู่ใน Netscape Enterprise Server, IBM Web Sphere หรืออาจจะเป็นส่วนที่เป็น Add-on ให้กับเว็บเซิร์ฟเวอร์ ก็ได้เช่น Apache Jserv, Tomcat, JRun หรือแม้กระทั่งเป็นส่วนหนึ่งที่อยู่ใน Application Server เช่น BEA Weblogic เป็นต้น ทั้งนี้การเลือกใช้ Servlet Engine แต่ละชนิดก็มักขึ้นอยู่กับปัจจัยหลายอย่างเช่น ความสะดวกในการรวมระบบที่จะสร้างขึ้นใหม่กับระบบที่มีอยู่แล้ว, งบประมาณที่มีอยู่สำหรับโครงการหรืออาจจะรวมไปถึงทักษะและประสบการณ์ส่วนตัวของนักพัฒนาแต่ละคน

6. การทำงานของ Servlet

สมมุติว่าเรามี interface หนึ่งชื่อ Servlet โดย interface นี้ประกอบไปด้วยฟังก์ชันสองฟังก์ชันดัง code snippet ข้างล่างนี้

```
interface Servlet {  
    public void init();  
    public void service();  
}
```

ถ้าเราต้องการสร้างคลาส ๆ หนึ่งชื่อ MyServlet โดยคลาสดังกล่าว implement *Servlet interface* คลาส MyServlet อาจจะเป็นอย่างข้างล่างนี้

```
class MyServlet implements Servlet {  
    public void init() {  
        System.out.println("Calling MyServlet init()...");  
    }  
    public void service() {  
        System.out.println("I'm MyServlet");  
    }  
}
```

เราจะสังเกตเห็นว่าคลาส MyServlet ทำการ implement ฟังก์ชัน init() และ service() ซึ่งเป็นฟังก์ชันที่ถูก define อยู่ใน Servlet interface

Servlet Engine ใช้ประโยชน์จากคอนเซ็ปของ interface ในการโหลด Servlet ต่าง ๆ ในช่วง Runtime โดย Servlet ต่าง ๆ จะถูกโหลดและ cast กลับไปเป็นอยู่ในรูปของ interface แทน

ถ้าดูจากตัวอย่างของเราหลังจาก MyServlet ถูก cast กลับไปเป็น instance ของ Servlet interface แล้วทาง Servlet Engine ก็จะสามารถเรียกฟังก์ชันอะไรก็ได้ที่ถูก define อยู่ใน Servlet interface (ในที่นี้ก็คือ init() และ

service()) ตัวอย่าง code ง่าย ๆ ที่ Servlet Engine ใช้ในการโหลด Servlet ในช่วง Runtime อาจจะเป็นอย่างข้างล่างนี้

```
public class SimpleServletEngine {  
    public static void main(String args[]) throws Exception {  
        if (args.length <= 0) {  
            System.out.println("Usage: java SimpleServletEngine javaClassName");  
            System.exit(0);  
        }  
        System.err.println("loading class " + args[0] + "...");  
        Class cls = Class.forName(args[0]);  
        Servlet servlet = (Servlet) cls.newInstance(); // casting  
        servlet.init();  
        servlet.service();  
    }  
}
```

ตัว Servlet ที่ Servlet Engine โหลดจะถูกอ้างอิงมาจาก argument ที่ชื่อ javaClassName โดย argument นี้จะถูกส่งผ่านไปให้ Class.forName() เพื่อเรียก Class object ที่จะใช้สร้าง Servlet instance ขึ้นมาโดยใช้คำสั่ง Class.newInstance() ซึ่งหลังจากนั้น Servlet instance ที่ได้จะถูก cast กลับให้กลายเป็น instance ของ Servlet interface เพื่อ Servlet Engine จะได้ใช้ในการเรียกฟังก์ชัน init() และ service()

เราอาจพูดง่าย ๆ ว่าหลักการทำงานของ Servlet Engine ก็คือการโหลดคลาสใดคลาสหนึ่งขึ้นมาโดยคลาสนั้นจะต้อง implement ตัว interface ที่ชื่อ Servlet ซึ่งทาง Servlet Engine ทราบอยู่แล้วว่ามีฟังก์ชันอะไรประกอบอยู่ใน Servlet interface บ้างโดยถ้า Service Engine ทำการ cast คลาสที่ implement Servlet interface (ในที่นี้ก็คือ MyServlet) กลับไปเป็น instance ของ Servlet interface แล้วตัว Servlet Engine ก็จะสามารถเรียกฟังก์ชันที่ชื่อ init() และ service() ที่ define อยู่ใน Servlet interface จากคลาสดังกล่าวได้

ในกรณีที่เราไม่ต้องการ implement ฟังก์ชันทุกฟังก์ชันที่ถูก define อยู่ใน Servlet interface เราก็อาจจะสร้างคลาสที่เป็น abstract ขึ้นมาคลาสหนึ่งก่อนโดยคลาสนี้จะทำการ implement ฟังก์ชันบางฟังก์ชันที่อยู่ใน Servlet interface แล้วให้ subclass ของ abstract คลาสนี้ทำการ implement ฟังก์ชันที่เหลือ ยกตัวอย่างเช่น

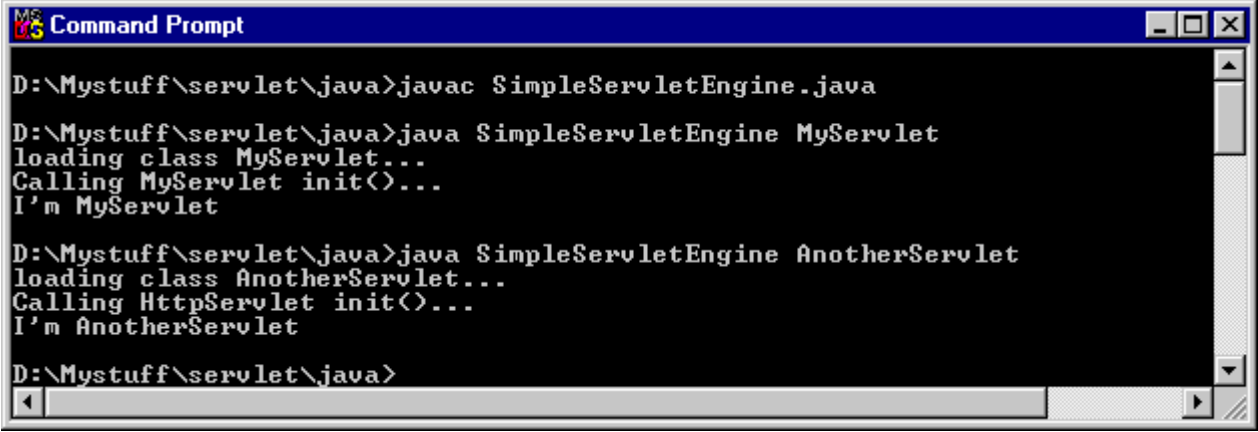
```
abstract class HttpServlet implements Servlet {  
    public void init() {  
        System.out.println("Calling HttpServlet init()...");  
    }  
}
```

```
// not implemented yet
public abstract void service();
}
```

ถ้าเราต้องการสร้าง Servlet ขึ้นมาโดยจะ implement แค่ฟังก์ชัน service() อย่างเดียว เราก็เพียง subclass คลาส HttpServlet เท่านั้น โดยคลาส Servlet ของเราอาจจะเป็นดังตัวอย่างข้างล่างนี้

```
class AnotherServlet extends HttpServlet {
    public void service() {
        System.out.println("I'm AnotherServlet");
    }
}
```

เราจะสังเกตเห็นว่าคลาส AnotherServlet จะไม่ต้องทำการ implement ฟังก์ชัน init() เพราะตัว parent คลาสของมันซึ่งก็คือ HttpServlet ได้ทำการ implement ไปแล้ว ในการโหลดคลาส AnotherServlet เข้าไปยัง Servlet Engine ก็ยังใช้หลักการเดียวกันกับคลาส MyServlet ข้างต้นคือโหลดคลาส AnotherServlet แล้ว cast กลับให้กลายเป็น instance ของ Servlet interface ซึ่งผลจากการรันโปรแกรม SimpleServletEngine โดยโหลด Servlet (หรือคลาส) ที่ชื่อ MyServlet และ AnotherServlet ก็จะออกมาเป็นดังรูป



```
MS Command Prompt
D:\Mystuff\servlet\java>javac SimpleServletEngine.java
D:\Mystuff\servlet\java>java SimpleServletEngine MyServlet
loading class MyServlet...
Calling MyServlet init()...
I'm MyServlet
D:\Mystuff\servlet\java>java SimpleServletEngine AnotherServlet
loading class AnotherServlet...
Calling HttpServlet init()...
I'm AnotherServlet
D:\Mystuff\servlet\java>
```

รูปที่ 2. การโหลด Servlet ต่าง ๆ ของ SimpleServletEngine

ภาคผนวก ข.

การติดตั้งเว็บไซต์ระบบให้บริการแผนที่บนเว็บ

1. การตั้งค่าเว็บเซิร์ฟเวอร์

โปรเว็บเป็นโปรแกรมที่ทำงานบนระบบปฏิบัติการวินโดวส์ (Microsoft Windows) ดังนั้นเราจึงเลือกใช้ Apache Web Server for Windows โดยดาวน์โหลดได้จาก

<http://www.apache.org/dist/httpd/binaries/win32/>

การตั้งค่าสำหรับโปรแกรม Apache จะสามารถแก้ค่าของเซิร์ฟเวอร์โดยการเข้าไปแก้ไขไฟล์ httpd.conf ในไดเรกทอรี bin โดยจะต้องเพิ่มคำสั่งเหล่านี้ลงไป เพื่อกำหนดพาทของโปรเว็บและพาทต่างๆที่จำเป็นในการใช้งาน โดย C:/WIN-PROLOG4040/PROWEB เป็นตำแหน่งของโปรเว็บโดยที่โปรเว็บต้องอยู่ในไดเรกทอรีของ WIN-PROLOG ด้วย

```
Alias /images "C:/WIN-PROLOG4040/PROWEB/IMAGES"
```

```
<Directory "C:/WIN-PROLOG4040/PROWEB/IMAGES">
```

```
Options Indexes FollowSymLinks MultiViews IncludesNoExec
```

```
AddOutputFilter Includes html
```

```
AllowOverride None
```

```
Order allow,deny
```

```
Allow from all
```

```
</Directory>
```

```
Alias /map "C:/WIN-PROLOG4040/PROWEB/HTML"
```

```
<Directory "C:/WIN-PROLOG4040/PROWEB/HTML">
```

```
Options Indexes FollowSymLinks MultiViews IncludesNoExec
```

```
AddOutputFilter Includes html
```

```
AllowOverride None
```

```
Order allow,deny
```

```
Allow from all
```

```
</Directory>
```

```
ScriptAlias /proweb/ "C:/WIN-PROLOG4040/PROWEB/"
```


ScriptAlias /ProWeb/ "C:/WIN-PROLOG4040/PROWEB/"

<Directory "C:/WIN-PROLOG4040/PROWEB">

AllowOverride None

Options None

Order allow,deny

Allow from all

</Directory>

2. การตั้งค่าโปรแกรมโปรเว็บ

1. คัดลอกไฟล์ pro386w.exe จากโปรแกรม WIN-PROLOG ลงไปในของไดเรกทอรีโปรเว็บ พร้อม
ทั้งเปลี่ยนชื่อไฟล์เป็น proweb.sys
2. คัดลอกไฟล์ DBLINK.PC ในไดเรกทอรี system ที่อยู่ในโปรแกรมโปรล็อกลงไปที่ไดเรกทอรี
proweb\system
3. คัดลอกไฟล์ LPADBW.DLL ในไดเรกทอรีโปรแกรมโปรล็อกลงไปที่ไดเรกทอรี proweb
4. คัดลอกไฟล์โปรล็อก(นามสกุล pl) ทั้งหมดไว้ใน proweb/examples
5. คัดลอกไฟล์ HTML ทั้งหมดไว้ใน proweb/html
6. คัดลอกไฟล์รูปภาพทั้งหมดไว้ใน proweb/images
7. เมื่อทำตามขั้นตอนนี้เสร็จแล้ว ให้เรียกไปที่ URL <http://localhost/map/index.html>